GeoClaw-claude

技术参考手册 / Technical Reference

v2.4.1

UrbanComp Lab · https://urbancomp.net

面向开发者：API / 架构 / 安全机制 / 扩展开发

项目	内容
文档类型	技术参考手册（开发者文档）
文档版本	v2.4.1
目标读者	Python 开发者 / 二次开发者
许可证	MIT License

目 录

第一章 系统架构概览

GeoClaw-claude 采用分层架构设计，从底层到顶层依次为：

```
+-----------------------------------------------------------+
|                CLI 层 (cli.py)                            |
|        ask / chat / memory / config / skill ...           |
+-----------------------------------------------------------+
|                NL 操作系统层                               |
|   GeoAgent -> NLProcessor -> NLExecutor -> LLMProvider    |
|               ContextCompressor                           |
+-----------------------------------------------------------+
|             Memory 记忆层                                  |
|   ShortTermMemory -> LongTermMemory -> MemoryArchive      |
|                                    -> VectorSearch        |
+-----------------------------------------------------------+
|                   GIS 分析层                              |
|   spatial_ops | network | raster_ops | mobility           |
|   io(vector/osm/remote) | cartography | coord_transform   |
+-----------------------------------------------------------+
|             安全层 (SecurityGuard)                         |
|  输出目录隔离 | 输入保护 | 路径穿越防护 | 系统目录保护  |
+-----------------------------------------------------------+
|             配置层 (Config)                                |
|   JSON 持久化 | 环境变量覆盖 | 类型推断              |
+-----------------------------------------------------------+
```

1.1 模块依赖关系

模块	直接依赖
cli.py	nl.agent, nl.processor, nl.executor, memory, updater, skill_manager
nl.agent	nl.processor, nl.executor, nl.context_compress
nl.executor	analysis.*, io.*, cartography.*, security, memory
nl.processor	nl.llm_provider, config
nl.llm_provider	anthropic, google.genai, openai (可选)
memory.manager	memory.short_term, memory.long_term, memory.archive, memory.vector_search
security	config
analysis.mobility	trackintel, geopandas, matplotlib

第二章 输出路径机制（v2.4.0 核心变更）

✦ v2.4.1 新特性

2.1 SecurityGuard 设计

SecurityGuard 是 GeoClaw-claude 的文件系统保护核心，负责确保所有写操作都在允许的 output_dir 下，同时保护输入文件（data_dir）不被意外覆盖。

```
from geoclaw_claude.security import SecurityGuard, get_guard

# 使用全局单例（基于 config.output_dir）
guard = get_guard()
safe_path = guard.safe_output_path("result.geojson")
# -> ~/geoclaw_output/result.geojson

# 创建自定义输出目录的独立实例（v2.4.0 新增机制）
guard = SecurityGuard(
    output_dir="/data/projects/wuhan",
    protected_dirs=["/data/raw_data"],
)
safe_path = guard.safe_output_path("hospitals_buffer.geojson")
# -> /data/projects/wuhan/hospitals_buffer.geojson

# 安全检查
resolved = guard.check_write("output.geojson")        # 返回安全绝对路径
guard.check_delete("output/old_result.geojson")       # 检查删除操作
guard.is_input_file("/data/raw_data/hospitals.geojson")  # -> True
```

2.2 动态 guard 实例机制

v2.4.0 的关键设计：NLExecutor 在 output_dir 参数非空时，为本次会话创建独立的 SecurityGuard 实例（而非修改全局单例），保证并发安全和实例隔离。

```
# v2.4.0 之前（所有调用共享全局 guard）
from geoclaw_claude.security import get_guard
safe_path = get_guard().safe_output_path(filename)

# v2.4.0 之后（NLExecutor 内部实现）
class NLExecutor:
    def __init__(self, ..., output_dir=None):
        # 若指定了输出目录，创建独立 guard（不影响全局单例）
        self._guard = None
        if output_dir:
            self._guard = SecurityGuard(
                output_dir=output_dir,
                protected_dirs=[cfg.data_dir],
            )

    def _get_safe_output_path(self, filename, subdir=""):
        if self._guard is not None:
            return str(self._guard.safe_output_path(filename, subdir=subdir))
        from geoclaw_claude.security import get_guard
```

```
        return str(get_guard().safe_output_path(filename, subdir=subdir))
```

2.3 SecurityGuard 完整 API 参考

方法签名	说明
SecurityGuard(output_dir, protected_dirs, verbose=False)	创建实例。output_dir: 允许写入目录；protected_dirs: 受保护输入目录列表
from_config(verbose=False) -> SecurityGuard	工厂方法：从 Config 读取目录配置，追加常见上传目录
check_write(path) -> Path	检查路径是否可安全写入。违规抛出 SecurityError，合规返回绝对路径
check_delete(path) -> Path	检查路径是否可安全删除
is_input_file(path) -> bool	判断路径是否在受保护的输入目录下
safe_output_path(filename, subdir='', auto_rename=True) -> Path	生成安全输出路径。auto_rename=True 时自动添加序号避免覆盖
redirect_to_output(path, subdir='') -> Path	将任意路径重定向到 output_dir 下，仅取文件名
protect_write(path_arg='path') -> decorator	函数装饰器：自动检查并重定向写入路径参数
get_audit_log() -> List[dict]	获取操作审计日志（含 time/action/path 字段）

第三章 NL 系统 API

3.1 GeoAgent

```
from geoclaw_claude.nl import GeoAgent

agent = GeoAgent(
    api_key    = None,     # Anthropic API Key（可选，优先于环境变量）
    use_ai     = None,     # True=AI, False=规则, None=自动
    verbose    = False,    # 打印调试信息
    session_id = None,     # 记忆会话 ID（None=自动生成）
    output_dir = None,     # 输出目录（None=使用 config.output_dir）  v2.4.0
)

reply = agent.chat("自然语言指令")    # 执行并返回回复文字
agent.end(title="会话标题")         # 结束会话，写入长期记忆
```

3.2 NLExecutor

```
from geoclaw_claude.nl import NLExecutor

exec = NLExecutor(
    memory_session = None,   # 绑定记忆会话 ID
    verbose        = True,   # 打印执行过程
    output_dir     = None,   # 输出目录 v2.4.0
)

# 执行 ParsedIntent
result = exec.execute_intent(intent)
print(result.success, result.message, result.result)

# 图层管理
exec.add_layer("hospitals", geo_layer)
layer = exec.get_layer("hospitals")
layers = exec.list_layers()    # -> ["hospitals", "buffer", ...]

# 输出路径（内部方法）
safe_path = exec._get_safe_output_path("result.geojson")
```

3.3 NLProcessor

```
from geoclaw_claude.nl import NLProcessor

proc = NLProcessor(
    api_key = None,    # AI API Key（自动从配置读取）
    use_ai  = None,    # True=AI 模式, False=规则模式, None=自动
    verbose = False,
)

intent = proc.parse("对医院做 1 公里缓冲区")
# intent.action     -> "buffer"
# intent.params     -> {"radius": 1000, "unit": "m"}
# intent.targets    -> ["hospitals"]
# intent.confidence -> 0.95
# intent.pipeline   -> False
```

3.4 LLMProvider

```
from geoclaw_claude.nl.llm_provider import LLMProvider, ProviderConfig

# 自动选择（按优先级：anthropicgemini->openai->qwen）
llm = LLMProvider.from_config()

# 显式指定
llm = LLMProvider(ProviderConfig(
    provider = "gemini",           # anthropic / gemini / openai / qwen
    api_key  = "AIza...",
    model    = "gemini-2.0-flash",
    base_url = "",                 # 仅 openai/qwen 需要
))

resp = llm.chat(
    system   = "你是 GIS 分析助手",
    messages = [{"role": "user", "content": "分析武汉市医院分布"}],
)
print(resp.content)
print(resp.provider, resp.model, resp.tokens_in, resp.tokens_out)
```

3.5 ProfileManager（v2.4.1 新增）

```
from geoclaw_claude.nl.profile_manager import ProfileManager, SoulConfig,
UserConfig

# 加载 soul.md 和 user.md（首次运行自动写入默认文件）
pm = ProfileManager(
    soul_path   = None,     # 自定义路径，None=~/.geoclaw_claude/soul.md
    user_path   = None,     # 自定义路径，None=~/.geoclaw_claude/user.md
    auto_create = True,     # 文件不存在时自动写入默认内容
).load()

# 生成 soul 行为边界 system prompt（高优先级）
soul_prompt = pm.build_system_prompt()

# 生成 user 偏好 context hint（软个性化）
user_hint   = pm.build_context_hint()

# 生成个性化欢迎语
welcome     = pm.build_welcome_message(mode="AI")

# 配置摘要
summary     = pm.summary()
# {"soul_loaded": True, "user_role": "geospatial analyst", "user_lang":
"zh", ...}

# 运行时热更新（重新读取文件）
pm.reload()

# 访问解析结果
soul: SoulConfig = pm.soul
user: UserConfig = pm.user

print(soul.identity)       # 系统身份描述
print(soul.principles)     # 核心原则列表
print(user.preferred_lang) # "zh" / "en" / "auto"
print(user.tool_prefs)     # ["QGIS", "GeoPandas", ...]
```

3.6 soul.md 与 user.md 文件格式

两个文件均为 Markdown 格式，首次运行时自动写入 ~/.geoclaw_claude/ 目录。soul.md 定义系统身份与安全边界，优先级高于 user.md。user.md 定义用户画像与长期偏好，不覆盖 soul 行为边界。

文件	优先级	作用	注入位置
soul.md	最高	系统身份、行为边界	LLM system prompt 前缀
user.md	较高	用户画像、长期偏好	LLM context hint

第四章 Memory 系统 API

4.1 四层记忆架构

```
ShortTermMemory    会话内操作日志 + 对象缓存（内存，会话结束即释放）
      | session end -> flush
      v
LongTermMemory     持久化 JSON 知识库    ~/.geoclaw_claude/memory/
      | auto-index
      v
MemoryArchive      完整会话快照存档      ~/.geoclaw_claude/archives/  [v2.4.0]
      | vectorize
      v
VectorSearch       TF-IDF/神经网络语义索引  ~/.geoclaw_claude/vector_index/
[v2.4.0]
```

4.2 MemoryManager（统一入口）

```
from geoclaw_claude.memory import get_memory

mem = get_memory()
mem.start_session("project_wuhan")

# 短期记忆
mem.log_op("buffer", "hospitals, 1km")
mem.remember("buf_layer", buf_layer)
mem.set_context("city", "wuhan")
mem.get_context("city")

# 长期记忆
mem.learn(title="武汉医院分布规律",
          content={"finding": "集中在三环内"},
          tags=["wuhan", "hospital"], importance=0.8)
results = mem.recall("武汉 医院")

mem.end_session(title="武汉医院分析复盘")
```

4.3 MemoryArchive（v2.4.0 新增）

✦ **v2.4.1 新特性**

方法	说明
save_session(title, ops_log, summary, tags, source) -> ArchiveEntry	存档会话。返回 ArchiveEntry（含 archive_id, date_str 等字段）
load(archive_id) -> ArchiveEntry	加载完整快照。entry.content['ops_log'] 获取内容
delete(archive_id) -> bool	删除存档
list_archives(source, tag, after, before) ->	列出存档（支持按来源/标签/时间范围过滤）

方法	说明
List[ArchiveEntry]	
search(query, top_k) -> List[ArchiveEntry]	关键词搜索存档标题和摘要
export(output_path) -> Path	全量导出为 JSON 文件（可跨机器迁移）
import_json(json_path, overwrite) -> int	从导出文件导入，返回导入条数
stats() -> dict	统计信息：total / size_human / sources 等

4.4 VectorSearch（v2.4.0 新增）

✦ **v2.4.1 新特性**

方法	说明
add(doc_id, text, title, tags, source, importance) -> None	添加文档到索引
remove(doc_id) -> bool	从索引删除文档
search(query, top_k, source_filter, tag_filter, min_score) -> List[SearchResult]	语义检索。SearchResult 含 doc_id / score / snippet / meta 字段
save() -> None	持久化索引到 ~/.geoclaw_claude/vector_index/
load() -> None	从磁盘加载索引
clear() -> None	清空索引
stats() -> dict	统计：documents / vocab_size / backend（tfidf or neural）

第五章 配置系统 API

```
from geoclaw_claude.config import Config

# 加载（优先级：环境变量 > config.json > 默认值）
cfg = Config.load()

# 常用字段（v2.4.0 新增字段用 v 标注）
cfg.anthropic_api_key   # Anthropic Claude API Key
cfg.anthropic_model     # 默认：claude-sonnet-4-20250514
cfg.gemini_api_key      # Google Gemini API Key  v
cfg.gemini_model        # 默认：gemini-2.0-flash  v
cfg.openai_api_key      # OpenAI API Key
cfg.openai_base_url     # 自定义 base_url
cfg.qwen_api_key        # 通义千问 API Key
cfg.llm_provider        # 强制 provider（空=自动）  v

cfg.ctx_max_tokens      # 触发压缩阈值（默认 6000）
cfg.ctx_target_tokens   # 压缩目标（默认 4000）
cfg.ctx_keep_recent     # 保留最近 N 条（默认 6）
cfg.ctx_compress_verbose

cfg.output_dir          # 分析结果输出目录（默认 ~/geoclaw_output）
cfg.data_dir            # 本地数据根目录
cfg.cache_dir           # 网络下载缓存
cfg.skill_dir           # 用户 Skill 目录
cfg.security_enabled    # 是否启用安全保护

# 修改单项并保存
cfg.set("output_dir", "/data/projects")
cfg.save()

cfg.ensure_dirs()      # 创建所有配置目录
cfg.summary()          # 返回配置摘要文字（隐藏 API Key）
```

5.1 所有支持的环境变量

环境变量	说明
GEOCLAW_OUTPUT_DIR	覆盖 output_dir（v2.4.0 新增支持）
GEOCLAW_ANTHROPIC_API_KEY	覆盖 anthropic_api_key
GEOCLAW_GEMINI_API_KEY	覆盖 gemini_api_key v
GEOCLAW_OPENAI_API_KEY	覆盖 openai_api_key
GEOCLAW_QWEN_API_KEY	覆盖 qwen_api_key
GEOCLAW_LLM_PROVIDER	覆盖 llm_provider v
GEOCLAW_<FIELD>	覆盖任意配置字段（大写，GEOCLAW_ 前缀）

第六章 Skill 系统开发指南

Skill 是用户自定义的 GIS 分析脚本，可通过 CLI 安装和执行。每个 Skill 是一个 Python 类，实现 run() 方法。

```
# 创建 Skill 模板
geoclaw-claude skill create my_analysis

# 安装本地 Skill
geoclaw-claude skill install path/to/my_analysis.py

# 运行 Skill
geoclaw-claude skill run my_analysis --data hospitals.geojson

# Skill 模板结构
class MyAnalysisSkill:
    name = "my_analysis"
    description = "我的自定义分析"
    version = "1.0.0"
    author = "UrbanComp Lab"

    def run(self, data=None, output="", use_ai=False, **kwargs):
        # data   : 输入文件路径或 GeoLayer
        # output : 输出路径（空=使用 config.output_dir）
        # use_ai : 是否启用 AI 辅助
        ...
        return {"status": "success", "result": result_layer}
```

第七章 版本历史

版本	变更摘要
v2.4.1 ✨	soul.md/user.md 个性化配置层（ProfileManager），GeoAgent 深度集成，profile CLI 命令组，196/196 测试
v2.4.0	Skill 安全审计（SkillAuditor，AST+正则，5 级风险），商场选址 Skill 双模式，Skill 编写规范文档
v2.3.1	输出路径配置（--output-dir / GEOCLAW_OUTPUT_DIR / Python API），文档重写
v2.3.0	Gemini API，MemoryArchive，VectorSearch，onboard 6 步向导，自动上下文压缩集成
v2.2.1	README 重组，移动性分析专节完善，NL 关键词映射修复
v2.2.0	武汉 GPS 轨迹 Demo 数据集（37,549 点），完整 Demo 脚本，trackintel 来源声明
v2.1.0	analysis/mobility/模块（trackintel 集成），NL 移动性操作支持（10 类）
v2.0.0	重大升级：自然语言 GIS 平台，多 Provider，安全机制
v1.3.0	自然语言操作系统（nl/模块，ask/chat 命令）
v1.2.0	自我检测与自动更新（check/update/self-check）
v1.1.0	Memory 记忆系统（短期+长期，memory CLI）
v1.0.0	首个正式版本：CLI、Skill 系统、路网/栅格分析

本文档由 UrbanComp Lab 编写。如有疑问，请访问 https://urbancomp.net